

InternSpark Internship Program

TASK 1

Build a RESTful API with Spring Boot

Candidate : Vasavi Thellajeera

Candidate ID : IS-2026-10396

Domain : Java Developer Intern

Mentor : Akash Dubey

Date: June 16, 2026

1. Objective

Build a fully functional RESTful API using Spring Boot that performs CRUD (Create, Read, Update, Delete) operations on a Student entity. The API uses an in-memory H2 database and exposes 5 REST endpoints tested with Postman.

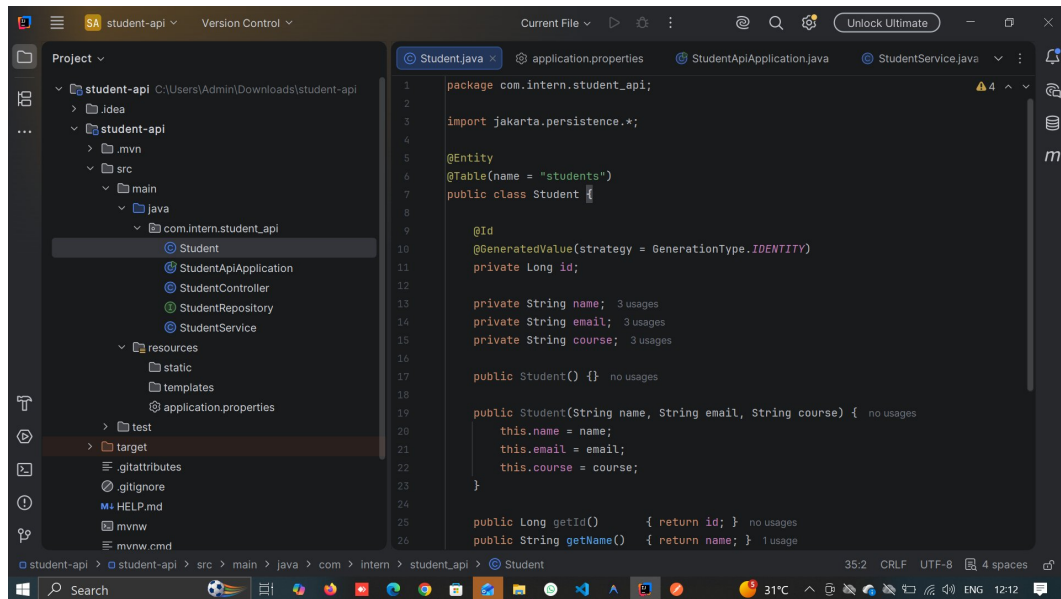
2. Technologies Used

Technology	Version / Details
Java (JDK)	17
Spring Boot	3.x
Maven	Build tool
Spring Web	REST controllers
Spring Data JPA	ORM layer
H2 Database	In-memory DB
IntelliJ IDEA	IDE
Postman	API testing

3. Project Structure

Four Java files created inside package **com.intern.student_api**:

```
student-api/
├── src/main/java/com/intern/student_api/
│   ├── StudentApiApplication.java ← Main entry point (auto-generated)
│   ├── Student.java               ← Entity / Data Model
│   ├── StudentRepository.java     ← Database interface (JPA)
│   ├── StudentService.java        ← Business logic
│   └── StudentController.java     ← REST API endpoints
└── src/main/resources/
    └── application.properties     ← H2 database configuration
```



Screenshot 1: IntelliJ IDEA – all 4 files visible in com.intern.student_api package

4. Source Code

4.1 Student.java – Entity

Represents the students table. JPA annotations map class fields to DB columns.

```
package com.intern.student_api;
import jakarta.persistence.*;

@Entity
@Table(name = "students")
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    private String course;

    public Student() {}
    public Student(String name, String email, String course) {
        this.name = name; this.email = email; this.course = course;
    }
    public Long getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public String getCourse() { return course; }
    public void setId(Long id) { this.id = id; }
    public void setName(String name) { this.name = name; }
    public void setEmail(String email) { this.email = email; }
    public void setCourse(String course) { this.course = course; }
}
```

4.2 StudentRepository.java – Data Access

Extends JpaRepository to auto-generate all SQL operations. Zero SQL needed.

```
package com.intern.student_api;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface StudentRepository extends JpaRepository<Student, Long> {
    // save(), findAll(), findById(), deleteById() provided automatically
}
```

4.3 StudentService.java – Business Logic

```
package com.intern.student_api;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.*;

@Service
public class StudentService {
    @Autowired
    private StudentRepository repo;

    public List<Student> getAllStudents() { return repo.findAll(); }
    public Optional<Student> getById(Long id) { return repo.findById(id); }
    public Student save(Student s) { return repo.save(s); }
```

```
public void delete(Long id)                { repo.deleteById(id); }

public Student update(Long id, Student newData) {
    Student s = repo.findById(id)
        .orElseThrow(() -> new RuntimeException("Student not found"));
    s.setName(newData.getName());
    s.setEmail(newData.getEmail());
    s.setCourse(newData.getCourse());
    return repo.save(s);
}
}
```

4.4 StudentController.java – REST Endpoints

```
package com.intern.student_api;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.*;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/students")
public class StudentController {
    @Autowired
    private StudentService service;

    @GetMapping                // GET /api/students
    public List<Student> getAll() { return service.getAllStudents(); }

    @GetMapping("/{id}")       // GET /api/students/{id}
    public ResponseEntity<Student> getById(@PathVariable Long id) {
        return service.getById(id).map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping              // POST /api/students
    public ResponseEntity<Student> create(@RequestBody Student s) {
        return ResponseEntity.status(HttpStatus.CREATED).body(service.save(s));
    }

    @PutMapping("/{id}")       // PUT /api/students/{id}
    public ResponseEntity<Student> update(@PathVariable Long id, @RequestBody Student s) {
        return ResponseEntity.ok(service.update(id, s));
    }

    @DeleteMapping("/{id}")    // DELETE /api/students/{id}
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        service.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

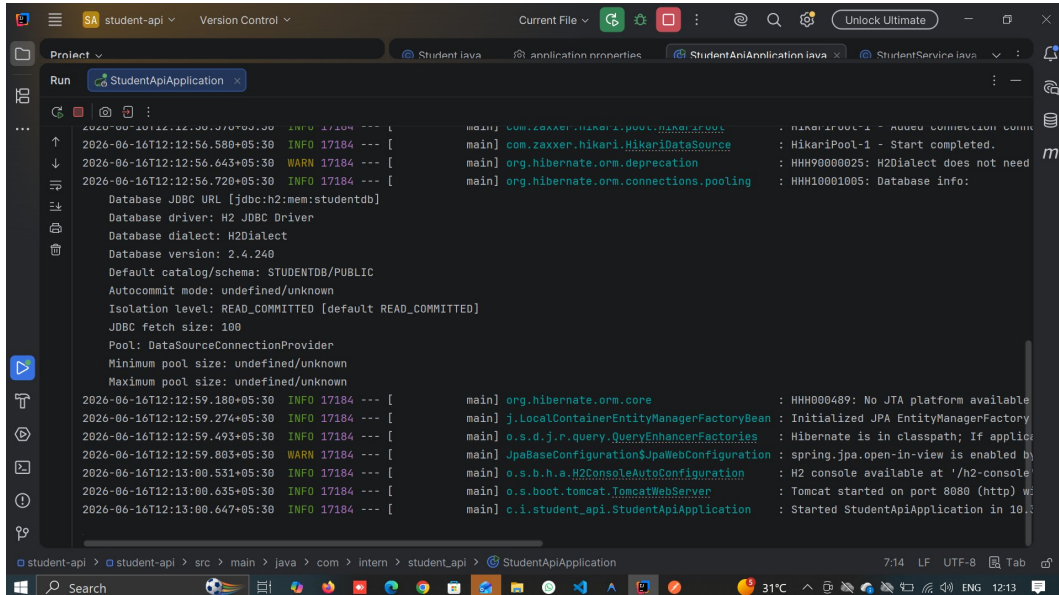
4.5 application.properties

```
spring.datasource.url=jdbc:h2:mem:studentdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=create-drop
spring.h2.console.enabled=true
```

5. Application Startup

Ran the app from IntelliJ IDEA by clicking  Run on StudentApiApplication.java.

- H2 database connected (jdbc:h2:mem:studentdb)
- Hibernate auto-created the 'students' table
- Tomcat started on port 8080
- Application started successfully



```
2026-06-16T12:12:56.707+05:30 INFO 17184 --- [main] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection conn
2026-06-16T12:12:56.580+05:30 INFO 17184 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2026-06-16T12:12:56.643+05:30 WARN 17184 --- [main] org.hibernate.orm.deprecation : HHH90000025: H2Dialect does not need
2026-06-16T12:12:56.720+05:30 INFO 17184 --- [main] org.hibernate.orm.connections.pooling : HHH10001005: Database info:
Database JDBC URL [jdbc:h2:mem:studentdb]
Database driver: H2 JDBC Driver
Database dialect: H2Dialect
Database version: 2.4.240
Default catalog/schema: STUDENTDB/PUBLIC
Autocommit mode: undefined/unknown
Isolation level: READ_COMMITTED [default READ_COMMITTED]
JDBC fetch size: 100
Pool: DataSourceConnectionProvider
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2026-06-16T12:12:59.180+05:30 INFO 17184 --- [main] org.hibernate.orm.core : HHH000489: No JTA platform available
2026-06-16T12:12:59.274+05:30 INFO 17184 --- [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory
2026-06-16T12:12:59.493+05:30 INFO 17184 --- [main] o.s.d.j.r.query.QueryEnhancerFactory : Hibernate is in classpath; If applic
2026-06-16T12:12:59.803+05:30 WARN 17184 --- [main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by
2026-06-16T12:13:00.531+05:30 INFO 17184 --- [main] o.s.b.h.a.H2ConsoleAutoConfiguration : H2 console available at '/h2-console'
2026-06-16T12:13:00.635+05:30 INFO 17184 --- [main] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) w
2026-06-16T12:13:00.647+05:30 INFO 17184 --- [main] c.i.student_api.StudentApiApplication : Started StudentApiApplication in 10.3
```

Screenshot 2: Console showing 'Tomcat started on port 8080' and app started successfully

6. API Testing with Postman

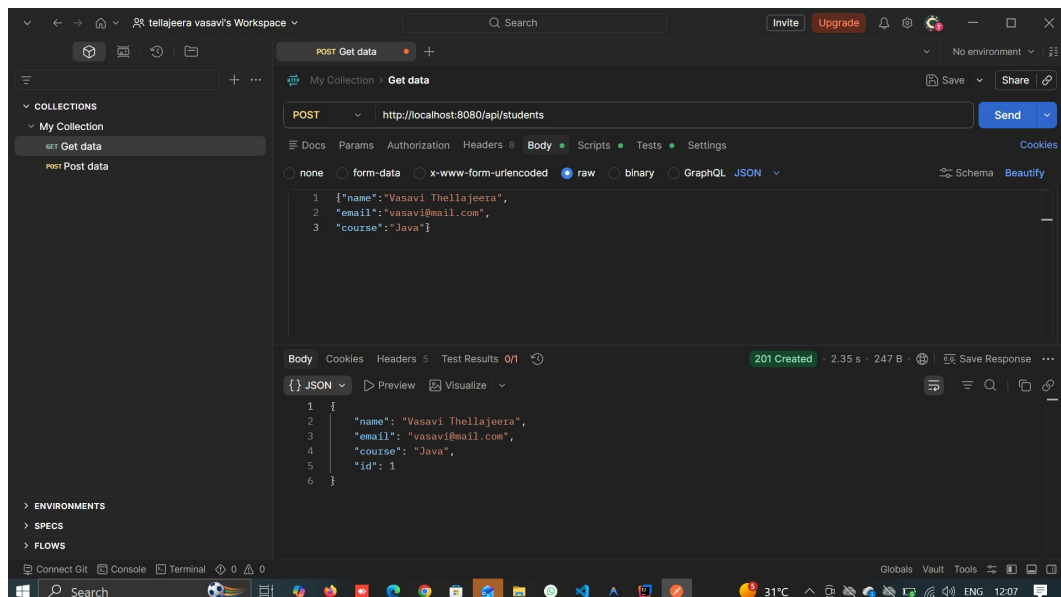
All 5 REST endpoints were tested using Postman:

#	Method	URL	Response	Status
1	POST	/api/students	Student JSON + id=1	201 Created ■
2	GET	/api/students	Array of students	200 OK ■
3	GET	/api/students/1	Single student JSON	200 OK ■
4	PUT	/api/students/1	Updated student JSON	200 OK ■
5	DELETE	/api/students/1	Empty body	204 No Content ■

Test 1 – POST /api/students (Create Student)

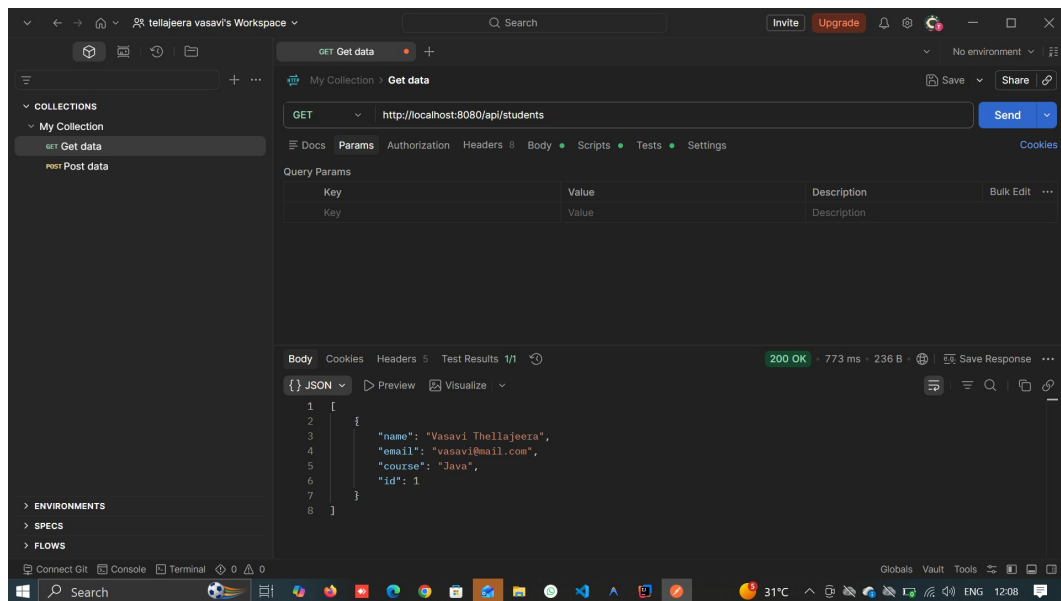
POST <http://localhost:8080/api/students>

Body (JSON): {"name":"Vasavi Thellajeera","email":"vasavi@mail.com","course":"Java"}



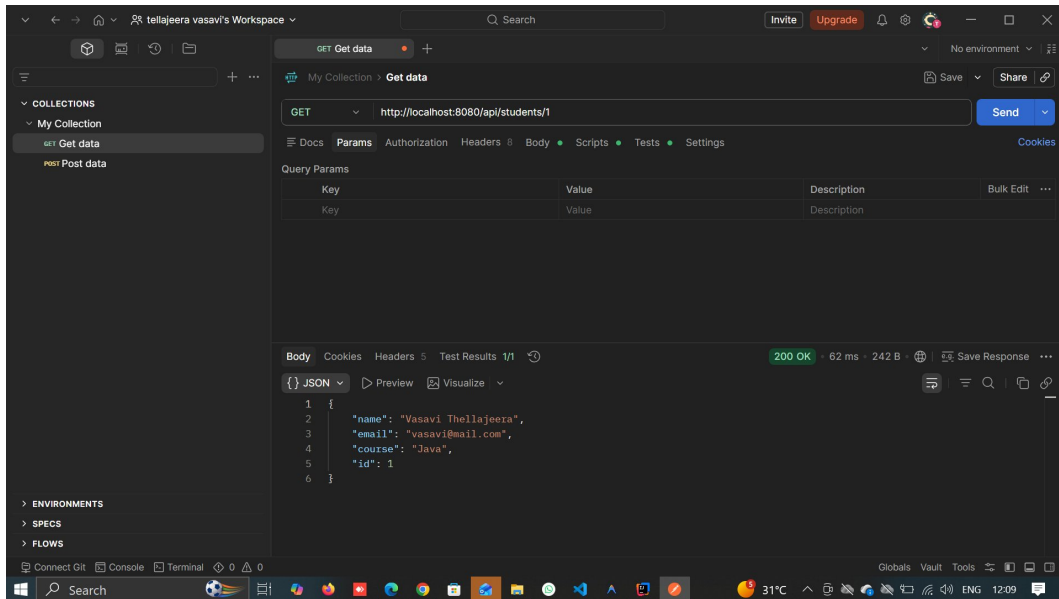
Screenshot 3: POST → 201 Created. Student created with id=1

Test 2 – GET /api/students (List All Students)



Screenshot 4: GET all → 200 OK. Returns array with 1 student

Test 3 – GET /api/students/1 (Get by ID)

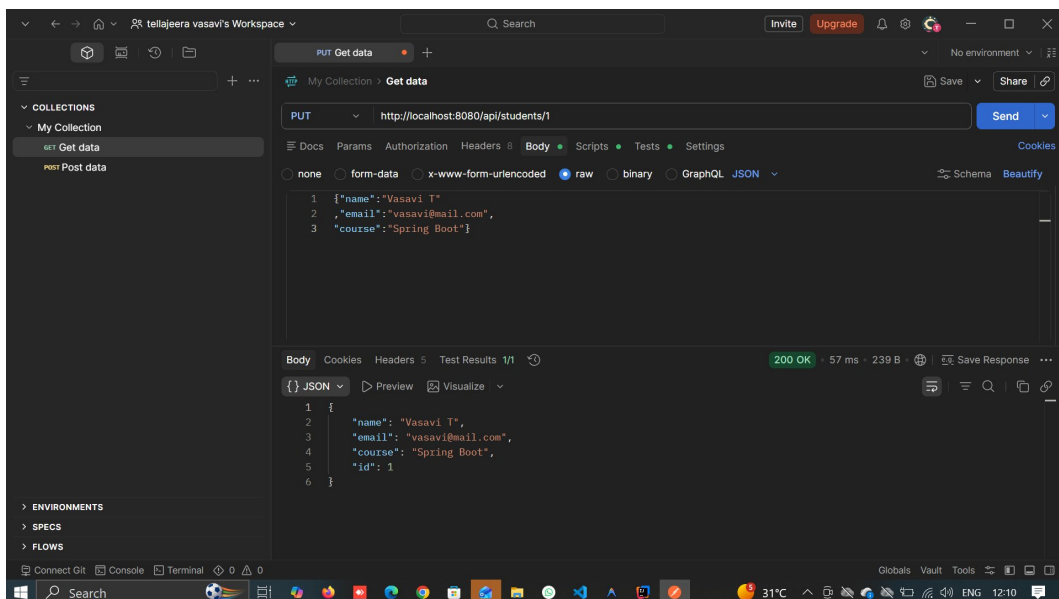


Screenshot 5: GET by id=1 → 200 OK. Returns single student object

Test 4 – PUT /api/students/1 (Update Student)

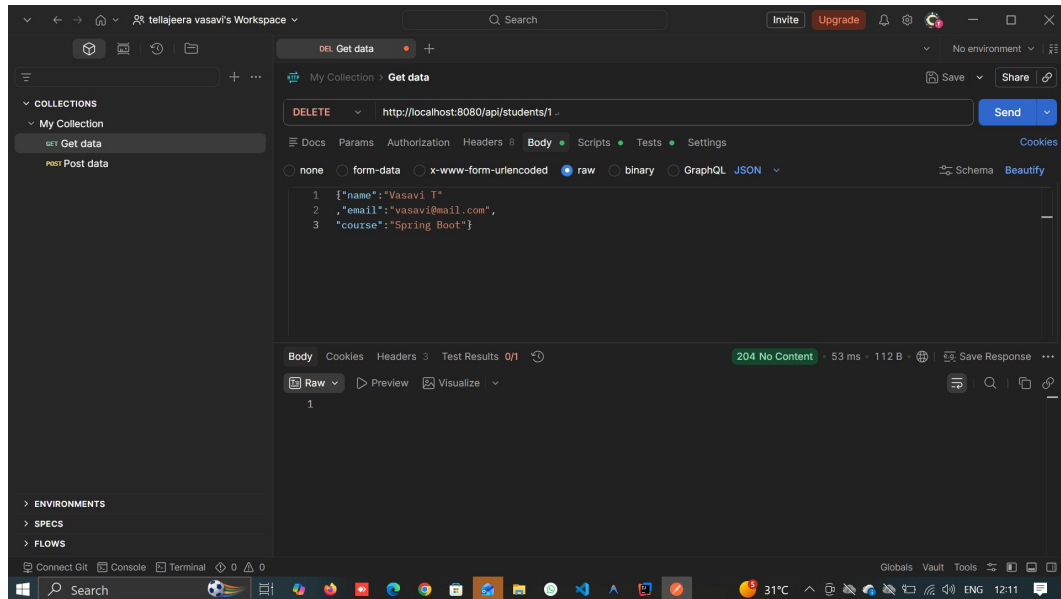
PUT <http://localhost:8080/api/students/1>

Body (JSON): {"name":"Vasavi T","email":"vasavi@mail.com","course":"Spring Boot"}



Screenshot 6: PUT → 200 OK. Name updated to "Vasavi T", course to "Spring Boot"

Test 5 – DELETE /api/students/1 (Delete Student)



Screenshot 7: DELETE → 204 No Content. Student successfully deleted (empty body)

7. Outcome & Learnings

Endpoint	Response	Result
POST /api/students	201 Created	■ Passed
GET /api/students	200 OK	■ Passed
GET /api/students/1	200 OK	■ Passed
PUT /api/students/1	200 OK	■ Passed
DELETE /api/students/1	204 No Content	■ Passed

- MVC architecture: Model (Student) → Repository → Service → Controller
- How @Entity, @Repository, @Service, @RestController work together
- HTTP methods: GET=Read, POST=Create, PUT=Update, DELETE=Remove
- Spring Data JPA auto-generates SQL from method names
- Testing REST APIs with Postman – JSON bodies and status codes
- In-memory H2 database configuration via application.properties